Modern C++ and Memory Safety:

WEEK 1:

In 2026, **move semantics** remain the bedrock of high-performance C++ distributed systems. While new standards like C++23 and C++26 have introduced higher-level abstractions (like Coroutines and Senders/Receivers), they all internally rely on move semantics to ensure "zero-copy" data transfers.

In a distributed context, where data often moves between network buffers, serialization engines, and application logic, `std::move` is the primary tool for avoiding the "double-allocation" tax.

Why Move Semantics are still Critical in 2026

- **Zero-Copy Networking:** High-performance libraries (like updated versions of gRPC or `asio`) use move semantics to pass ownership of `std::vector<char>` or `std::string` buffers from the network driver directly to your business logic without copying the bytes.
- **Coroutine Efficiency:** C++20/23 coroutines rely heavily on moving "promises" and "task" objects. Without move semantics, the overhead of suspending and resuming distributed tasks would be prohibitive.
- **The "Movable-Only" Pattern:** In distributed systems, resources like unique network connections or file descriptors are often modeled as `std::unique_ptr` or custom handles. These **must** be moved because they cannot be logically copied.

Modern Distributed Systems Context (C++23/26)

While `std::move` is still used, the *way* we use it has evolved. In 2026, you'll see it most often in:

1. **Serialized Buffer Handoffs:** Moving a large FlatBuffers or Protobuf object into a message queue.
2. **Sender/Receiver Pattern (`std::execution`):** C++26's major addition. It uses move semantics to chain asynchronous operations across different nodes or threads without data duplication.
3. **Trivial Relocatability:** Although "destructive move" was a hot topic for C++26, much of the performance gain is currently handled by compilers optimizing `std::move` for types that are "trivially relocatable" (bits can just be `memcpy`'d).

Implementation Example

Here is a typical 2026-style pattern using your template: moving a heavy data payload into a distributed task queue.

```C++
#include <bits/stdc++.h>
using namespace std;
struct Payload {
    vector<int> data;
    string metadata;
    // Move constructor ensuring no-copy transfer
    Payload(vector<int>&& d, string&& m) : data(std::move(d)), metadata(std::move(m)) {}
};
```

```
void process_locally(Payload p) {
    // Logic for handling the distributed packet
    cout << "Processing " << p.data.size() << " elements.\n";
}

void Solve(){
    int n; cin >> n;
    vector<int> vec(n);
    for(int i = 0; i < n; i++) cin >> vec[i];

    // In distributed systems, we 'move' the buffer to the sender
    // to avoid O(N) copy overhead.
    Payload p(std::move(vec), "sensor_data_v1");

    // vec is now in a "moved-from" (empty) state.
    process_locally(std::move(p));
}

int32_t main(){
    ios::sync_with_stdio(0); cin.tie(0);
    int t = 1;
    cin >> t;
    while(t--) {
        Solve();
    }
    return 0;
}
```

input.in

Plaintext

```
1

5

10 20 30 40 50
```

output.out

Plaintext

```
Processing 5 elements.
```

Move semantics is fundamentally about **transferring ownership** of resources (like memory, file handles, or network sockets) from one object to another, rather than making expensive copies.

Think of it this way:

- **Copy:** You have a house. I want a house exactly like yours. I build a brand new house, brick by brick, looking identical to yours. (Slow, expensive).
- **Move:** You are leaving the country and don't need your house anymore. You give me the keys. I now own the house. You have nothing (but you are technically still "alive"). (Fast, cheap).

Here is an in-depth breakdown.

1. The Core Mechanics: Lvalues vs. Rvalues

To understand Move, you must understand value categories:

- **Lvalue (Left Value):** An object that has a name and persists beyond a single expression (e.g., `vector<int> a;`). You *cannot* safely steal from it because the user might use `a` later.
- **Rvalue (Right Value):** A temporary object that does not have a name or is about to be destroyed (e.g., `vector<int>{1, 2, 3}` or the return value of a function). You *can* safely steal from it because nobody else can reference it.

2. What is `std::move`?

`std::move` is a lie. It **does not move anything**. It does not generate any executable code that moves data.

It is strictly a **cast**. It takes an Lvalue (something you can't steal from) and casts it to an Rvalue Reference (`T&&`), effectively telling the compiler: *"I promise I won't use this object again. You have my permission to plunder its resources."*

3. Deep Dive Example

We will create a custom `Wrapper` class that manages a raw pointer to demonstrate exactly how the "theft" happens compared to a copy.

Scenario: We are managing a heavy array.

C++

```
#include <bits/stdc++.h>
using namespace std;
class Buffer {
public:
    int* data;
    int size;

    // 1. Constructor
    Buffer(int n) {
        size = n;
        data = new int[n];
        for(int i=0; i<n; i++) data[i] = i * 10;
        cout << "[Default Construct] allocated " << size << " ints.\n";
    }

    // 2. Destructor
    ~Buffer() {
        if (data != nullptr) {
            cout << "[Destruct] freeing memory.\n";
            delete[] data;
```

```
        } else {
            cout << "[Destruct] nothing to free (was moved).\n";
        }
    }

    // 3. COPY Constructor (Deep Copy - Slow)
    Buffer(const Buffer& other) {
        size = other.size;
        data = new int[size]; // ALLOCATION! Heavy operation.
        memcpy(data, other.data, size * sizeof(int));
        cout << "[COPY] Deep copied data.\n";
    }

    // 4. MOVE Constructor (Shallow Copy - Fast)
    // Note the 'noexcept'. This is critical for STL containers to use move.
    Buffer(Buffer&& other) noexcept {
        // STEAL the resource
        data = other.data;
        size = other.size;

        // NULLIFY the source so its destructor doesn't kill the data we just stole
        other.data = nullptr;
        other.size = 0;

        cout << "[MOVE] Stole pointer. Source is now empty.\n";
    }
};

void Solve(){
    cout << "--- Create A ---\n";
    Buffer a(5);

    cout << "\n--- Copy A to B ---\n";
    // Calls Copy Constructor because 'a' is an lvalue
    Buffer b = a;

    cout << "\n--- Move A to C ---\n";
    // std::move(a) casts 'a' to 'Buffer&&' (rvalue reference).
    // This triggers the Move Constructor.
    Buffer c = std::move(a);

    cout << "\n--- End of Scope ---\n";
}

//|---------------------------------------[MAIN]---------------------------------------|
int32_t main(){
  auto begin = std::chrono::high_resolution_clock::now();
  ios::sync_with_stdio(0); cin.tie(0);
  int t = 1;
  // cin>> t; // Commented out to run single example flow
  for(int i = 1; i <= t; i++)
    {
        Solve();
    }
  return 0;
}
```

input.in

(Empty or dummy input, as this example logic is self-contained)

output.out

```
Plaintext
```

--- Create A ---

[Default Construct] allocated 5 ints.

--- Copy A to B ---

[COPY] Deep copied data.

--- Move A to C ---

[MOVE] Stole pointer. Source is now empty.

--- End of Scope ---

[Destruct] freeing memory.

[Destruct] freeing memory.

[Destruct] nothing to free (was moved).

Important Notes & Facts

1. **The `noexcept` Rule:**
   Always mark your move constructors `noexcept`. If `std::vector` needs to resize, it will move elements to the new memory block *only if* the move constructor is `noexcept`. If it's not marked, `std::vector` will fall back to **Copying** everything to guarantee exception safety (Strong Exception Guarantee).
2. **Valid but Unspecified State:**
   After `std::move(x)`, `x` is not necessarily "empty." It is in a "valid but unspecified" state. You can destroy it, or you can assign a new value to it, but you should not read its current value.
3. **`const` blocks Move:**
   If you have `const string s = "hello";`, `std::move(s)` will **not** move. It will cast it to `const string&&`. Since a move constructor requires a modifiable reference

(string&&) to "steal" and nullify the source pointers, it cannot bind to const. It will silently fall back to the **Copy Constructor**.

4. **The Rule of Five:**
   If you implement one of the following, you likely need to implement all five to handle resource management correctly:
   - Destructor
   - Copy Constructor
   - Copy Assignment Operator
   - Move Constructor
   - Move Assignment Operator
5. **Small String Optimization (SSO):**
   For small strings (usually < 15 or 22 chars), std::string stores data directly in the handle, not on the heap. Moving a small string acts like a copy (copying bytes) because there is no heap pointer to steal. Move semantics shines primarily with heap-allocated resources.

---

1. std::unique_ptr: The Sole Owner

Concept:

std::unique_ptr represents **exclusive ownership**. It is a smart pointer that says, "I own this object, and no one else does."

- When the unique_ptr goes out of scope, it automatically deletes the object it points to.
- **Copying is banned:** You cannot copy a unique_ptr (because two things cannot uniquely own the same object).
- **Moving is allowed:** You can transfer ownership using std::move.

Performance:

It has practically **zero overhead** compared to a raw pointer. It is just a raw pointer wrapper with a destructor.

Key Operations:

- std::make_unique<T>(args): The safest way to create one.
- ptr.get(): Access the raw pointer (for observing, not owning).
- ptr.release(): Give up ownership (returns the raw pointer, doesn't destroy it).
- ptr.reset(): Destroys the current object and optionally takes a new one.

Implementation Example

In this example, we simulate a system processing "Tasks." A task can only be owned by one worker at a time. We will move tasks between vectors to demonstrate ownership transfer.

C++

```cpp
/**
 * author: Anicetus_7
 * created: 2026-02-03 20:20:00
**/
#include <bits/stdc++.h>
using namespace std;

struct Task {
    int id;
    string name;

    Task(int i, string n) : id(i), name(n) {
        cout << "[Construct] Task " << id << " (" << name << ") created.\n";
    }

    ~Task() {
        cout << "[Destruct] Task " << id << " destroyed.\n";
    }

    void print() {
        cout << " -> Working on: " << name << "\n";
    }
};

// Function taking ownership (Sink)
// The unique_ptr is passed by value (move required), consuming the caller's pointer.
void consume_task(unique_ptr<Task> t) {
    cout << "Inside worker function:\n";
    t->print();
    // 't' dies here, destroying the Task
}

void Solve(){
    int n;
    cin >> n; // Number of tasks

    // Vector of unique_ptrs.
    // Commonly used for polymorphic collections or managing hefty resources.
    vector<unique_ptr<Task>> vec;

    for(int i = 0; i < n; i++) {
        string s; cin >> s;
        // make_unique is cleaner than 'new'
        vec.push_back(make_unique<Task>(i+1, s));
    }

    cout << "\n--- Transferring Ownership ---\n";
```

```
    // We want to process the first task.
    // unique_ptr<Task> copy = vec[0]; // ERROR: Call to implicitly-deleted copy constructor

    // We must MOVE it. vec[0] becomes null (empty).
    unique_ptr<Task> tmp = std::move(vec[0]);

    if(vec[0] == nullptr) {
        cout << "Vector index 0 is now empty (moved from).\n";
    }

    cout << "\n--- Passing to Function ---\n";
    // Moving 'tmp' into the function. 'tmp' loses ownership.
    consume_task(std::move(tmp));

    if(tmp == nullptr) {
        cout << "tmp variable is now empty (moved into function).\n";
    }

    cout << "\n--- Cleaning up remaining vector ---\n";
    // The vector destructor will destroy all remaining unique_ptrs automatically.
}

//|---------------------------------------[MAIN]---------------------------------------|
int32_t main(){
  auto begin = std::chrono::high_resolution_clock::now();
  ios::sync_with_stdio(0); cin.tie(0);
  int t = 1;
  cin >> t;
  for(int i = 1; i <= t; i++) {
      Solve();
  }
  auto end = std::chrono::high_resolution_clock::now();
  auto elapsed = std::chrono::duration_cast<std::chrono::nanoseconds>(end - begin);
  cerr << "Time measured: " << elapsed.count() * 1e-9 << " seconds.\n";
  return 0;
}
```

input.in

```
Plaintext
```

```
1
3
DataProcess
ImageRender
LogWrite
```

output.out

```
Plaintext

[Construct] Task 1 (DataProcess) created.
[Construct] Task 2 (ImageRender) created.
[Construct] Task 3 (LogWrite) created.

--- Transferring Ownership ---
Vector index 0 is now empty (moved from).

--- Passing to Function ---
Inside worker function:
 -> Working on: DataProcess
[Destruct] Task 1 destroyed.
tmp variable is now empty (moved into function).

--- Cleaning up remaining vector ---
[Destruct] Task 2 destroyed.
[Destruct] Task 3 destroyed.
```

2. std::shared_ptr: Cooperative Ownership

Concept:

`std::shared_ptr` represents **shared ownership**. It says, "I own this object, but others might own it too."

- The object is destroyed **only when the last** `shared_ptr` pointing to it is destroyed or reset.
- **Copying is allowed:** When you copy a `shared_ptr`, a "reference count" (ref count) is incremented.
- **Destruction:** When a `shared_ptr` dies, it decrements the ref count. If the count hits 0, the object is deleted.

Performance Cost:

Unlike `unique_ptr`, `shared_ptr` has overhead:

1. **Control Block:** It allocates a separate "Control Block" on the heap to store the reference count.
2. **Atomic Operations:** Incrementing/decrementing the ref count is thread-safe (atomic), which is slightly slower than normal arithmetic.

Key Operations:

- `std::make_shared<T>(args)`: **Crucial.** Allocates the object AND the control block in *one* memory chunk (better cache locality and performance than `new`).
- `ptr.use_count()`: Returns the number of owners (mostly for debugging).

`shared_ptr` Example

Here we simulate a file that is accessed by multiple processes simultaneously. The file should remain open as long as *at least one* process is holding it.

C++

```cpp
#include <bits/stdc++.h>
using namespace std;
struct SharedFile {
   string name;
   SharedFile(string n) : name(n) {
      cout << "  [OPEN] File '" << name << "' opened.\n";
   }

   ~SharedFile() {
      cout << "  [CLOSE] File '" << name << "' closed (Ref Count hit 0).\n";
   }
};

void Solve(){
   // process 1 creates the file
   cout << "--- Create Shared Ptr (P1) ---\n";
   shared_ptr<SharedFile> p1 = make_shared<SharedFile>("config.json");
   cout << "Ref Count: " << p1.use_count() << "\n";

   {
      cout << "\n--- P2 joins and shares ownership ---\n";
      shared_ptr<SharedFile> p2 = p1; // Copy allowed! Ref count increases.
      cout << "Ref Count: " << p1.use_count() << "\n";

      cout << "\n--- P3 joins ---\n";
      shared_ptr<SharedFile> p3 = p1;
      cout << "Ref Count: " << p1.use_count() << "\n";

      cout << "\n--- P2 and P3 die (End of inner scope) ---\n";
   } // p2 and p3 go out of scope here.

   cout << "Ref Count: " << p1.use_count() << "\n";

   cout << "\n--- P1 dies (End of Function) ---\n";
} // p1 dies, count -> 0, file closed.

int32_t main(){
  ios::sync_with_stdio(0); cin.tie(0);
  Solve();
  return 0;
}
```

output.out

Plaintext

--- Create Shared Ptr (P1) ---

[OPEN] File 'config.json' opened.

Ref Count: 1

--- P2 joins and shares ownership ---

Ref Count: 2

--- P3 joins ---

Ref Count: 3

--- P2 and P3 die (End of inner scope) ---

Ref Count: 1

--- P1 dies (End of Function) ---

[CLOSE] File 'config.json' closed (Ref Count hit 0).

---

3. std::weak_ptr: The Observer

Concept:

std::weak_ptr is a "non-owning" reference to an object managed by shared_ptr.

- It does **not** increase the reference count.
- It is used to break **Circular Dependencies** (The "Deadly Embrace").
- You cannot access the object directly. You must convert it to a shared_ptr (using .lock()) to check if the object still exists.

The Problem: Cyclic Dependency

If Object A has a shared_ptr to B, and Object B has a shared_ptr to A, their reference counts will never reach zero. They will keep each other alive forever, causing a memory leak.

The Solution:

Make one of the pointers a weak_ptr.

weak_ptr Example (Breaking Cycles)

We will create a graph node structure. A Parent owns Children, but Children only *observe* their Parent (to avoid a cycle).

C++

```cpp
#include <bits/stdc++.h>
using namespace std;
struct Node {
    string value;
    vector<shared_ptr<Node>> children;
    weak_ptr<Node> parent; // <--- WEAK POINTER (prevents cycle)

    Node(string v) : value(v) { cout << "  [Create] " << value << "\n"; }
    ~Node() { cout << "  [Destroy] " << value << "\n"; }
};

void Solve(){
    cout << "--- Setup Cycle ---\n";
    shared_ptr<Node> root = make_shared<Node>("Root");
    shared_ptr<Node> child = make_shared<Node>("Child");

    // Root owns Child
    root->children.push_back(child);

    // Child observes Root (Weakly)
    // If this was shared_ptr, neither would ever be destroyed!
    child->parent = root;

    cout << "\n--- Checking Parent from Child ---\n";
    if (auto p = child->parent.lock()) { // .lock() promotes weak to shared temporarily
        cout << "Child says: My parent is " << p->value << "\n";
    } else {
        cout << "Child says: My parent is gone.\n";
    }

    cout << "\n--- Exiting Scope ---\n";
} // Root goes out of scope -> Count 0 -> Destroy Root -> Child ref count drops -> Destroy Child.

//|---------------------------------------[MAIN]---------------------------------------|
int32_t main(){
  ios::sync_with_stdio(0); cin.tie(0);
  Solve();
  return 0;
}
```

output.out

Plaintext

--- Setup Cycle ---

[Create] Root

[Create] Child

--- Checking Parent from Child ---

Child says: My parent is Root

--- Exiting Scope ---

[Destroy] Root

[Destroy] Child

Summary Cheat Sheet

Pointer Type	Ownership	Copyable?	Overhead	Use Case
`unique_ptr`	Exclusive	**No** (Move only)	Zero	Default choice. Factory functions, unique resources.
`shared_ptr`	Shared (Ref Counted)	**Yes**	Low (Control Block)	Shared resources, DAGs (nodes with multiple parents).
`weak_ptr`	None (Observer)	**Yes**	Low	Caches, breaking circular references in Graphs/Trees.

A coding problem where using `weak_ptr` is the only way to avoid

Memory Limit Exceeded (MLE) caused by leaks

Here is a practical, production-style example demonstrating **std::weak_ptr acting as a Cache**.

This is a very common pattern in distributed systems (like the ones you asked about earlier). You want to hold a reference to an object only if it is currently being used by someone else. If no one is using it, the cache should not keep it alive artificially.

The "Self-Cleaning" Cache Pattern

Scenario: We have a WidgetFactory. Creating a Widget is expensive (simulated by a heavy load).

1. We return std::shared_ptr<Widget> to users.
2. The factory keeps a std::weak_ptr<Widget> to track it.
3. If a user asks for the same widget while another user is holding it, we return the existing one (fast).
4. If the previous users have finished (ref count drops to 0), the widget is destroyed. The factory sees the weak_ptr has expired and re-creates it (slow).

C++

```
#include <iostream>

#include <memory>

#include <string>

#include <unordered_map>

// A heavy resource we want to share

class Widget {

public:

  Widget(const std::string& id) : m_id(id) {

    std::cout << "  [Resource] Loading heavy widget: " << m_id << " (Expensive)\n";

  }


  ~Widget() {

    std::cout << "  [Resource] Unloading widget: " << m_id << "\n";

  }


  void do_something() const {
```

```
        std::cout << "    -> Widget " << m_id << " is active.\n";
    }

private:
    std::string m_id;
};

class WidgetFactory {
public:
    // Returns a shared_ptr to a widget.
    // If it exists in memory, returns the existing one.
    // If not, loads a new one.
    std::shared_ptr<Widget> get_widget(const std::string& id) {
        // 1. Check if we have a weak record of this ID
        if (m_cache.find(id) != m_cache.end()) {
            std::weak_ptr<Widget> weak_ref = m_cache[id];

            // 2. Try to lock it (convert weak -> shared)
            if (std::shared_ptr<Widget> existing = weak_ref.lock()) {
                std::cout << "  [Factory] Cache HIT for " << id << ". Returning existing instance.\n";
                return existing;
            } else {
                std::cout << "  [Factory] Cache MISS for " << id << " (Object expired).\n";
                // Clean up the dead key from the map
                m_cache.erase(id);
            }
        } else {
            std::cout << "  [Factory] Cache MISS for " << id << " (First time access).\n";
```

```
        }
        // 3. Create fresh instance
        std::shared_ptr<Widget> new_widget = std::make_shared<Widget>(id);
        // 4. Store a weak reference in the cache
        m_cache[id] = new_widget;
        return new_widget;
    }

private:
    // We map IDs to weak pointers.
    // We do NOT want the factory to own the widgets.
    std::unordered_map<std::string, std::weak_ptr<Widget>> m_cache;
};

int main() {
    WidgetFactory factory;

    {
        std::cout << "--- Client A requests 'Config' ---\n";
        std::shared_ptr<Widget> userA = factory.get_widget("Config");
        userA->do_something();

        std::cout << "\n--- Client B requests 'Config' (A is still holding it) ---\n";
        // Should hit the cache
        std::shared_ptr<Widget> userB = factory.get_widget("Config");
        userB->do_something();
```

```
        std::cout << "\n--- Client A releases 'Config' ---\n";
        userA.reset(); // A drops ownership, but B still holds it
        std::cout << "(Widget is still alive because B holds it)\n";

    } // Client B goes out of scope here. Ref count -> 0. Widget is destroyed.

    std::cout << "\n--- Both clients finished. Widget should be destroyed. ---\n";
    std::cout << "--- Client C requests 'Config' ---\n";

    // Should miss the cache (expired) and reload
    std::shared_ptr<Widget> userC = factory.get_widget("Config");
    userC->do_something();

    return 0;
}
```

Output Analysis

```
Plaintext
```

```
--- Client A requests 'Config' ---
  [Factory] Cache MISS for Config (First time access).
  [Resource] Loading heavy widget: Config (Expensive)
    -> Widget Config is active.

--- Client B requests 'Config' (A is still holding it) ---
  [Factory] Cache HIT for Config. Returning existing instance.
    -> Widget Config is active.
```

--- Client A releases 'Config' ---

(Widget is still alive because B holds it)

[Resource] Unloading widget: Config

--- Both clients finished. Widget should be destroyed. ---

--- Client C requests 'Config' ---

[Factory] Cache MISS for Config (Object expired).

[Resource] Loading heavy widget: Config (Expensive)

-> Widget Config is active.

Why `std::weak_ptr` is mandatory here?

If `m_cache` stored `std::shared_ptr<Widget>`, the factory itself would count as an owner. The Reference Count would **never** drop to zero because the Factory map would always hold a copy. The Widgets would never be unloaded, effectively causing a memory leak (or at least, an infinite cache that eats all RAM).

`std::weak_ptr` allows the Factory to know *about* the Widget without *keeping it alive*.

How Lambda Captures interact with `unique_ptr` (using `std::move` inside the capture list)

This is a very frequent pattern in modern C++ (C++14 and later), especially in asynchronous programming or when dispatching tasks to threads.

The Problem

A lambda in C++ is essentially an object (an anonymous struct). When you "capture" variables, you are actually creating member variables inside that struct.

Since `std::unique_ptr` **cannot be copied**, you cannot capture it by value `[ptr]` or by implicit value `[=]`. Both try to invoke the (deleted) copy constructor.

The Solution: Init Capture

C++14 introduced **Init Capture** (Generalized Lambda Capture). This allows you to define a *new* member variable inside the lambda and initialize it arbitrarily—including by moving an

existing variable into it.

The syntax looks like this:

`[ new_name = std::move(old_name) ]`

Example: The "Fire and Forget" Task

In this example, we create a heavy task and immediately hand it off to a lambda. The lambda takes full ownership of the pointer.

```
#include <iostream>
#include <memory>
#include <string>
#include <vector>
#include <functional>

class HeavyRequest {
public:
    HeavyRequest(int id) : m_id(id) {
        std::cout << "  [Construct] Request " << m_id << " allocated.\n";
    }
    ~HeavyRequest() {
        std::cout << "  [Destruct] Request " << m_id << " deallocated.\n";
    }
    void process() const {
        std::cout << "  [Process] Handling Request " << m_id << "...\n";
    }
private:
    int m_id;
};

int main() {
    std::cout << "--- 1. Create a unique_ptr in main scope ---\n";
    std::unique_ptr<HeavyRequest> req = std::make_unique<HeavyRequest>(101);

    std::cout << "\n--- 2. Create Lambda and MOVE ownership into it ---\n";

    // CAPTURE SYNTAX EXPLAINED:
    // [ captured_req = std::move(req) ]
    // 'captured_req' is the name of the variable INSIDE the lambda.
    // 'req' is the variable OUTSIDE being moved from.
    auto task_lambda = [captured_req = std::move(req)]() {
        std::cout << "    (Inside Lambda) I own the object now.\n";
        captured_req->process();
    };

    // At this point, 'req' in main is effectively nullptr.
    if (!req) {
        std::cout << "(Main) 'req' is now empty/null.\n";
    }

    std::cout << "\n--- 3. Execute Lambda ---\n";
    task_lambda(); // The lambda destroys 'captured_req' when it goes out of scope?
                   // No, the lambda object itself holds it.
```

```
    std::cout << "\n--- 4. End of Main ---\n";
    // task_lambda goes out of scope here.
    // It's destructor runs, which destroys 'captured_req'.
    return 0;
}
```

Output

```
Plaintext

--- 1. Create a unique_ptr in main scope ---
  [Construct] Request 101 allocated.

--- 2. Create Lambda and MOVE ownership into it ---
(Main) 'req' is now empty/null.

--- 3. Execute Lambda ---
    (Inside Lambda) I own the object now.
  [Process] Handling Request 101...

--- 4. End of Main ---
  [Destruct] Request 101 deallocated.
```

Important Fact: `mutable`

By default, the `operator()` of a lambda is `const`. This means you cannot modify the members (the captured variables) inside the lambda.

- **Calling methods on `unique_ptr`:** You **can** call `captured_req->process()` without `mutable` because `operator->` on a `unique_ptr` is a const method (it doesn't change the pointer itself, just accesses what it points to).
- **Modifying the `unique_ptr` itself:** If you wanted to reset the pointer *inside* the lambda, you must mark the lambda as `mutable`.

```
C++

auto cleaner = [ptr = std::move(my_ptr)]() mutable {
    ptr.reset(); // Allowed only because of 'mutable'
};
```

std::thread: The Move-Only Execution Agent

Just like std::unique_ptr, **std::thread is a move-only class**.

1. **It cannot be copied:** You cannot have two std::thread objects representing the *same* physical OS thread. Who would be responsible for joining it? It would be chaos.
2. **It must be moved:** If you want to put a thread into a container (like std::vector) or return it from a factory function, you must move it.

Furthermore, **passing arguments** to a thread often requires move semantics if those arguments are heavy or unique (like unique_ptr).

Scenario 1: Passing unique_ptr INTO a Thread

This is a classic compile error for beginners. std::thread tries to **copy** its arguments into the new thread's stack space by default. If you pass a unique_ptr, the copy fails. You **must** move it.

```
#include <iostream>
#include <thread>
#include <vector>
#include <memory>
#include <string>
#include <chrono>

struct Request {
    int id;
    std::string data;

    Request(int i, std::string d) : id(i), data(d) {}
    ~Request() { std::cout << "  [Request " << id << "] Destroyed (Memory freed).\n"; }
};

// Worker function that takes OWNERSHIP of the request
void server_handler(std::unique_ptr<Request> req) {
    // Simulate work
    std::this_thread::sleep_for(std::chrono::milliseconds(100));
    std::cout << "  [Thread " << std::this_thread::get_id() << "] Processed: " << req->data << "\n";
    // 'req' is destroyed here when the function ends
}

int main() {
    std::cout << "--- 1. Creating Request ---\n";
    auto req = std::make_unique<Request>(1, "UserLogin_Payload");

    std::cout << "--- 2. Spawning Thread (Moving Argument) ---\n";

    // ERROR: std::thread t(server_handler, req);
    // ^ This tries to COPY 'req'. unique_ptr cannot be copied.

    // CORRECT: Move 'req' into the thread constructor
    std::thread worker(server_handler, std::move(req));
```

```
    if (!req) {
        std::cout << "(Main) 'req' pointer is now null. The thread owns it.\n";
    }

    // We must wait for the thread to finish
    worker.join();
    std::cout << "--- 3. Thread Joined. Exiting. ---\n";

    return 0;
}
```

Output:

```
--- 1. Creating Request ---
--- 2. Spawning Thread (Moving Argument) ---
(Main) 'req' pointer is now null. The thread owns it.
  [Thread 140239] Processed: UserLogin_Payload
  [Request 1] Destroyed (Memory freed).
--- 3. Thread Joined. Exiting. ---
```

Scenario 2: Moving std::thread itself (Vector of Threads)

In distributed systems, you rarely have just one thread. You usually have a pool. Since std::thread cannot be copied, you cannot populate a std::vector<std::thread> using copy assignment.

C++

```
#include <iostream>
#include <thread>
#include <vector>
#include <mutex>

std::mutex cout_mutex; // To prevent garbled text

void worker_task(int id) {
    std::lock_guard<std::mutex> lock(cout_mutex);
    std::cout << "Worker " << id << " is running.\n";
}

int main() {
    int num_threads = 3;
    std::vector<std::thread> pool;

    // Optimization: avoid resizing during push_backs
    pool.reserve(num_threads);
```

```
    std::cout << "--- Spawning Threads ---\n";
    for(int i = 0; i < num_threads; i++) {
        // Method A: Construct directly in place (Implicit Move)
        // pool.emplace_back(worker_task, i);

        // Method B: Create named object, then MOVE it into vector
        std::thread t(worker_task, i);

        // pool.push_back(t); // ERROR: Cannot copy 't' into vector
        pool.push_back(std::move(t)); // CORRECT: Steal 't's handle into vector
    }

    std::cout << "--- Threads are running independently ---\n";

    // Joining
    for(auto& t : pool) {
        if(t.joinable()) t.join();
    }

    std::cout << "--- All Done ---\n";
    return 0;
}
```

Important Warning: std::ref

Sometimes, you do **not** want to move *or* copy. You want the thread to modify a variable that lives in main().

Since std::thread defaults to "decay-copying" (copying the value) to ensure thread safety, passing a reference normally doesn't work as expected. You must use the std::ref() wrapper.

```
void modifier(int& n) { n += 10; }

int x = 5;
std::thread t(modifier, std::ref(x)); // Passes actual reference to x
t.join();
// x is now 15
```

Note: This is dangerous! If x goes out of scope before the thread finishes, the thread crashes (Dangling Reference).

This concludes the core pillars of Move Semantics:

1. **Resources** (std::unique_ptr)
2. **Shared Resources** (std::shared_ptr)
3. **Observation** (std::weak_ptr)
4. **Execution** (std::thread and Lambdas)

Would you like a single **Master Challenge Problem** (a clear specification) that requires you to combine all these concepts (Unique, Shared, Weak, and Threads) to solve a Distributed Cache simulation?

The Master Challenge: The "Ghost" Cache System

This problem is designed to force you to use **Unique**, **Shared**, and **Weak** pointers in their correct semantic contexts, along with **Threads** for asynchronous tasks.

Scenario: You are building the core engine for a high-performance distributed cache node. This node handles data storage, asynchronous logging, and "ghost" observers that watch data without keeping it alive.

The Specifications

1. **The Store (`shared_ptr`):**
   - The main cache is a `map<string, shared_ptr<string>>`.
   - Data stored here is "owned" by the cache.
2. **The Ghost Watchers (`weak_ptr`):**
   - Clients can "watch" a key. This creates a `weak_ptr<string>` pointing to the cache entry.
   - Store these watchers in a `vector<pair<string, weak_ptr<string>>>`.
   - Crucially, a watcher **must not** prevent the cache from deleting the data. If the cache deletes the key, the watcher should detect that the data is gone (the "ghost" has vanished).
3. **The Async Logger (`unique_ptr` + `thread`):**
   - When a specific "heavy" log command is received, you must create a `unique_ptr<string>` containing the log message.
   - You must **move** this `unique_ptr` into a new thread.
   - The thread takes ownership, "processes" it (prints it), and then exits. This simulates a fire-and-forget background task where memory safety is guaranteed by `unique_ptr`.

Input/Output Format (for your `input.in`)

Commands:

- `STORE <key> <val>`: Create a `shared_ptr` and store it in the main map.
- `WATCH <key>`: Create a `weak_ptr` from the existing `shared_ptr` and add it to the watchers list. If the key doesn't exist, then ignore.
- `DELETE <key>`: Remove the key from the main map (destroying the `shared_ptr`).

- `STATUS`: Iterate through all **watchers**. Try to `lock()` them. If valid, print `"<key>: Alive: <val>"`. If expired, print `"<key>: Expired"`.
- `ASYNC_LOG <msg>`: Create a `unique_ptr`, spawn a thread, move the ptr to the thread. The thread must print `LOG: <msg>`. (Note: To ensure deterministic output for your `output.out` check, join this thread immediately after spawning, though in production you would detach).

```
/**

* author: Anicetus_7

* created: 2026-02-04 18:00:00

**/

#include <bits/stdc++.h>

#include <thread>

#include <mutex>

#include <memory>

#include <print>

using namespace std;



// Global storage for the simulation

map<string, shared_ptr<string>> cache;

vector<pair<string, weak_ptr<string>>> watchers;

mutex cout_mutex; // For thread-safe printing if you detach threads
```

```
// Helper to simulate thread work
void process_log(unique_ptr<string> msg_ptr) {
   // TODO: Acquire lock on cout_mutex if necessary
   // Print "LOG: " + *msg_ptr
   lock_guard<mutex> lock(cout_mutex);
   cout << "LOG: " + *msg_ptr;
}


void Solve(){
   cache.clear();
   watchers.clear();


   int q;
   cin >> q;
   while(q--) {
       string cmd;
       cin >> cmd;


       if (cmd == "STORE") {
           string key, val;
           cin >> key >> val;
```

```
            // TODO: Create a shared_ptr<string> and store in 'cache'
            shared_ptr<string> tmp = make_shared<string>(val);
            cache[key] = tmp;
        } else if (cmd == "WATCH") {
            string key;
            cin >> key;
            // TODO: Look up key in cache.
            // If found, create a weak_ptr from the shared_ptr and push
{key, wp} to 'watchers'.
            if(cache.find(key) != cache.end()){
              shared_ptr<string> tmp = cache[key];
              weak_ptr<string> wp = tmp;
              watchers.push_back({key, wp});
            }
        }
        else if (cmd == "DELETE") {
            string key;
            cin >> key;
            // TODO: Erase key from 'cache'
            if(cache.find(key) != cache.end()){
              cache.erase(key);
```

```
            }

        }

        else if (cmd == "STATUS") {

            // TODO: Iterate 'watchers'.

            // Use wp.lock() to check if alive.

            // Print "<key>: Alive: <val>" or "<key>: Expired"

            for(auto p: watchers){

              string key  = p.first;

              weak_ptr<string> ptr = p.second;

              if(auto sp = ptr.lock()){

                cout << key << ": Alive: "<< *sp<< endl;

              }else cout << key << ": Expired: "<<endl;

            }

        }

        else if (cmd == "ASYNC_LOG") {

            string msg;

            cin >> msg;

            // TODO: Create unique_ptr<string>.

            // Spawn a std::thread passing process_log and moving the
unique_ptr.

            // Join the thread immediately (for deterministic output in
```

```
this problem).

            auto uptr = make_unique<string>(msg);

            thread t(process_log, std::move(uptr));

            if(t.joinable())t.join();

        }

    }

}



//|-----------------------------------------[MAIN]-----------------------
--------------------|

int32_t main(){

 int t = 1;

 cin>> t;

 for(int i = 1; i <= t; i++)

    {

        //cout << "Case #" << i << ": \n";

        Solve();

    }

 return 0;

}
```

WEEK 2:

This is where you bridge the gap between "solving algorithmic puzzles" and "writing high-performance infrastructure."

In competitive programming, if you have a massive vector, you pass it by `const &` to avoid copying. But what if a function *creates* a massive vector and needs to return it? Or what if you want to transfer ownership of a 1GB data buffer from a network thread to a worker thread? A deep copy would kill your server's latency.

Move Semantics solve this by stealing the memory pointers instead of copying the data.

Here is your comprehensive, day-by-day breakdown for Week 2, strictly using modern, production-grade C++.

Day 1 & 2: The Foundation – L-values vs. R-values

Before you can move data, you must understand how the compiler views variables.

- **L-value (Locator Value):** An object that occupies an identifiable location in memory (it has an address). You can safely take its address using `&`.
    - *Example:* `int x = 10;` (`x` is an L-value).
- **R-value (Read Value):** A temporary object. It does not have a persistent memory address. It is usually destroyed at the end of the expression.
    - *Example:* `int x = 10 + 5;` (`10 + 5` evaluates to `15`, which is a temporary R-value).

The R-value Reference (`&&`)

In Modern C++, you can bind a reference specifically to a temporary object using `&&`. This tells the compiler: *"I know this object is about to be destroyed, so I am going to cannibalize its resources."*

C++

```
#include <iostream>
```

```cpp
#include <string>

void printValue(const std::string& str) {
    std::cout << "L-value reference called: " << str << "\n";
}

void printValue(std::string&& str) {
    std::cout << "R-value reference called (Temporary): " << str << "\n";
}

int main() {
    std::string name = "Anicetus";

    printValue(name);          // Calls L-value version (name is persistent)
    printValue("Temporary!");  // Calls R-value version (string literal is temporary)

    return 0;
}
```

Day 3: The Illusion of `std::move`

`std::move` is the most misunderstood function in C++. **It does not move anything.** `std::move()` is simply a cast. It unconditionally casts an L-value into an R-value reference (`&&`). It tells the compiler: *"Treat this persistent variable as if it were a temporary variable, so its resources can be stolen."*

C++

```cpp
#include <iostream>
```

```cpp
#include <vector>
#include <utility> // Required for std::move

int main() {
    std::vector<int> source = {1, 2, 3, 4, 5};

    // We cast 'source' to an R-value.
    // The vector constructor sees the R-value and STEALS the internal pointers.
    std::vector<int> destination = std::move(source);

    std::cout << "Destination size: " << destination.size() << "\n"; // Outputs 5
    std::cout << "Source size: " << source.size() << "\n";         // Outputs 0 (It was hollowed out!)

    return 0;
}
```

Day 4 & 5: The Rule of Five & Custom Move Constructors

If you are writing a class that manages a raw resource (like a heap-allocated array, a file descriptor, or a network socket), you must implement the **Rule of Five**:

1. Destructor
2. Copy Constructor
3. Copy Assignment Operator
4. Move Constructor
5. Move Assignment Operator

A **Move Constructor** does two things:

1. **Shallow Copy:** It copies the raw pointer from the source to the destination.
2. **Nullify:** It sets the source's pointer to `nullptr` so the source's destructor doesn't delete

the memory we just stole.

The Weekend Assignment: "The String Class Clone"

This is your crucible. You will write your own basic string class that manages a heap-allocated character array. This proves you understand exactly how memory ownership transfers.

Your Goal: Implement the missing Rule of Five methods.

Here is your starter code. Put this in a modern C++ environment and fill in the `// TODO` sections.

C++

```
#include <iostream>
#include <cstring>
#include <vector>
#include <chrono>

class MyString {
private:
    char* data;
    size_t length;

public:
    // 1. Default Constructor
    MyString() : data(nullptr), length(0) {}

    // 2. Parameterized Constructor
    MyString(const char* str) {
        length = std::strlen(str);
```

```
    data = new char[length + 1];
    std::strcpy(data, str);
    std::cout << "Constructed: " << data << "\n";
}

// 3. Destructor
~MyString() {
    delete[] data;
}

// ========================================================================
// TODO 1: Implement the Copy Constructor (Deep Copy)
// Signature: MyString(const MyString& other)
// ========================================================================

// ========================================================================
// TODO 2: Implement the Move Constructor (Steal the pointer)
// Signature: MyString(MyString&& other) noexcept
// Hint: Don't forget to set other.data to nullptr!
// ========================================================================

// ========================================================================
// TODO 3: Implement the Copy Assignment Operator (Deep Copy)
// Signature: MyString& operator=(const MyString& other)
// Hint: Check for self-assignment (if this == &other)
// ========================================================================
```

```
    // ================================================================
    // TODO 4: Implement the Move Assignment Operator (Steal the pointer)
    // Signature: MyString& operator=(MyString&& other) noexcept
    // ================================================================

    void print() const {
        if (data) std::cout << data << "\n";
        else std::cout << "[Empty/Moved]\n";
    }
};

int main() {
    std::cout << "--- Testing Copy vs Move ---\n";

    MyString str1("Hello Backend World");

    std::cout << "\nAttempting Copy:\n";
    MyString str2 = str1; // Should trigger Copy Constructor

    std::cout << "\nAttempting Move:\n";
    MyString str3 = std::move(str1); // Should trigger Move Constructor

    std::cout << "\nFinal States:\n";
    std::cout << "str1: "; str1.print(); // Should be empty
    std::cout << "str2: "; str2.print(); // Should have "Hello Backend World"
```

```
    std::cout << "str3: "; str3.print(); // Should have "Hello Backend World"


    return 0;
}
```

Why the `noexcept` keyword?

You'll notice `noexcept` on the move methods. In C++, containers like `std::vector` will only use your Move Constructor during a reallocation if they are *guaranteed* it won't throw an exception. If you omit `noexcept`, `std::vector` will quietly fall back to using your slow Copy Constructor to ensure strong exception safety.

Would you like me to provide the solution for the `// TODO` blocks, or do you want to write it yourself and paste it here for a code review?

This week, you will leave behind the brittle "competitive programming" way of handling data and errors. In CP, if a function cannot find a valid answer, returning `-1`, `INF`, or an empty array is fine. In a distributed system, returning a magic number is how you cause a billion-dollar production outage.

Week 3 focuses on **Modern Error Handling (`std::optional`, `std::variant`)** and **Asynchronous Callbacks (Lambdas and `std::function`)**.

Day 1: Modern Error Handling (`std::optional`)

You need to express the concept of "value or no value" without using pointers (which introduce memory management risks and reference semantics) or sentinel values (like `-1`).

`std::optional<T>` gives you **value semantics**. It allocates zero heap memory; the internal value is stored directly inside the `std::optional` object alongside a boolean flag indicating its presence.

C++

```cpp
#include <iostream>
#include <optional>
#include <string>

std::optional<int> safeDivide(int a, int b) {
    if (b == 0) {
        return std::nullopt; // Explicitly returning "nothing"
    }
    return a / b;
}

int main() {
    auto result = safeDivide(10, 2);

    // std::optional implicitly converts to bool to check for presence
    if (result) {
        std::cout << "Result: " << *result << "\n"; // Safe to dereference
    }

    // value_or() provides a clean fallback without writing if-else chains
    auto badResult = safeDivide(10, 0);
    std::cout << "Bad result fallback: " << badResult.value_or(-999) << "\n";

    return 0;
}
```

Day 2: Type-Safe Unions (`std::variant`)

Backend systems constantly process dynamic data schemas (e.g., parsing a JSON field that could be an integer or a string). C-style `union` types are completely unsafe because they don't track which type is currently active.

`std::variant<Types...>` is a type-safe union. Under the hood, its memory footprint is the size of its largest alternative type, plus a small internal tag to track the active type, plus padding for alignment.

C++

```
#include <iostream>
#include <variant>
#include <string>

using ConfigValue = std::variant<int, double, std::string>;

int main() {
    ConfigValue val = 42; // Currently holds an int

    if (std::holds_alternative<int>(val)) {
        std::cout << "Integer: " << std::get<int>(val) << "\n";
    }

    val = "localhost"; // Now holds a string (automatically destroys the int safely)

    // std::get throws std::bad_variant_access if you request the wrong active type
    try {
        std::cout << "Float: " << std::get<double>(val) << "\n";
    } catch (const std::bad_variant_access& e) {
```

```
        std::cout << "Error caught: Not a double!\n";
    }

    return 0;
}
```

Day 3 & 4: Lambdas Under the Hood & Captures

Lambdas are not magic functions. When you compile a lambda, the C++ compiler automatically generates an anonymous, hidden class (a closure object) that overloads `operator()`.

When you specify a **capture list**, you are telling the compiler to generate private member variables inside that hidden class.

- **Capture by value `[x]`**: The compiler copies `x` into the closure object.
- **Capture by reference `[&x]`**: The compiler stores a reference.

The `mutable` trap: By default, the `operator()` generated for a lambda is `const`. You cannot modify state captured by value. To modify the closure's internal copy, you must explicitly mark it `mutable`.

C++

```
#include <iostream>
#include <vector>

int main() {
    int counter = 0;

    // 'mutable' allows us to modify the captured-by-value 'counter'
    // This modifies the lambda's internal copy, NOT the 'counter' in main.
    auto statefulLambda = [counter]() mutable {
```

```
    counter++;
    return counter;
  };

  std::cout << "Lambda call 1: " << statefulLambda() << "\n"; // Prints 1
  std::cout << "Lambda call 2: " << statefulLambda() << "\n"; // Prints 2
  std::cout << "Original main counter: " << counter << "\n";  // Prints 0

  return 0;
}
```

Day 5: Callbacks with `std::function`

When building a multithreaded server (which you will do in Phase 2), you need to pass lambdas into thread pools or event loops. `std::function` is a polymorphic wrapper that can store *any* callable target as long as the signature matches.

C++

```
#include <iostream>
#include <functional>

// Accepts any callable that takes no arguments and returns void
void executeAsyncCallback(const std::function<void()>& callback) {
  std::cout << "[System] Executing callback...\n";
  callback();
}

int main() {
```

```
    int connectionId = 8080;


    executeAsyncCallback([connectionId]() {
        std::cout << "Successfully bound to port: " << connectionId << "\n";
    });


    return 0;
}
```

The Weekend Assignment: "The Safe Config Parser"

Your Task: Write a robust, memory-safe configuration manager.

1. Create a class `ConfigParser`.
2. It must maintain a private map: `std::unordered_map<std::string, std::variant<int, std::string, bool>>`.
3. Implement a method `std::optional<std::variant<int, std::string, bool>> getValue(const std::string& key)`. If the key doesn't exist, return `std::nullopt` (do not throw an exception, and do not return a dummy value).
4. Implement a `filterConfigs` method that accepts a `std::function<bool(const std::string&)>` (a predicate lambda) and prints only the keys that satisfy the lambda's condition.

Constraints: You must compile cleanly with `-Wall -Wextra -Werror`. No raw pointers, no macros.

[std::optional In C++17]

This video provides an excellent visual breakdown of the performance characteristics and safety guarantees of replacing traditional pointers with standard optionals.

The Weekend Assignment: "The String Class Clone"

This is your crucible. You will write your own basic string class that manages a heap-allocated character array. This proves you understand exactly how memory ownership transfers.

Your Goal: Implement the missing Rule of Five methods.

```
#include <iostream>
#include <cstring>
#include <vector>
#include <chrono>

class MyString {
private:
    char* data;
    size_t length;

public:
    // 1. Default Constructor
    MyString() : data(nullptr), length(0) {}

    // 2. Parameterized Constructor
    MyString(const char* str) {
        length = std::strlen(str);
        data = new char[length + 1];
        std::strcpy(data, str);
        std::cout << "Constructed: " << data << "\n";
    }

    // 3. Destructor
    ~MyString() {
```

```
        delete[] data;
    }

    //
==========================================================================
=
    // TODO 1: Implement the Copy Constructor (Deep Copy)
    // Signature: MyString(const MyString& other)
    //
==========================================================================
=
    MyString(const MyString& other){
        this->length = other.length;
        if(other.data){
            data = new char[length+1];
            std::strcpy(data, other.data);
        }else data = nullptr;
        std::cout << "Copy constructor triggered " <<std::endl;
    }


    //
==========================================================================
=
    // TODO 2: Implement the Move Constructor (Steal the pointer)
    // Signature: MyString(MyString&& other) noexcept
    // Hint: Don't forget to set other.data to nullptr!
    //
==========================================================================
=
```

```
    MyString(MyString&& other) noexcept : data(other.data), length(other.length){
        other.data= nullptr;
        other.length = 0;
        std::cout<< "Move constructor triggered"<<std::endl;
    }
    //
==========================================================================
=
    // TODO 3: Implement the Copy Assignment Operator (Deep Copy)
    // Signature: MyString& operator=(const MyString& other)
    // Hint: Check for self-assignment (if this == &other)
    //
==========================================================================
=
    MyString& operator=(const MyString& other){
        if(this != &other){ // self assignment guard
            length = other.length;
            if(other.data){
                data = new char[length+1];
                std::strcpy(data, other.data);
            }else data= nullptr;
        }
        std::cout << "Copy assignment triggered" <<std::endl;
        return *this;
    }
    //
==========================================================================
```

```
=
    // TODO 4: Implement the Move Assignment Operator (Steal the pointer)
    // Signature: MyString& operator=(MyString&& other) noexcept
    //
==========================================================================
=
    MyString& operator=(MyString&& other) noexcept {
        if(this != &other){ // self assignment guard
            delete[] data;// clean existing


            data = other.data;
            length = other.length;


            // nullify the source
            other.data= nullptr;
            other.length = 0;
        }
        std::cout << "Move Assignment triggered" <<std::endl;
        return *this;
    }
    void print() const {
        if (data) std::cout << data << "\n";
        else std::cout << "[Empty/Moved]\n";
    }
};
```

```
int main() {
    std::cout << "--- Testing Copy vs Move ---\n";

    MyString str1("Hello Backend World");

    std::cout << "\nAttempting Copy:\n";
    MyString str2 = str1; // Should trigger Copy Constructor

    std::cout << "\nAttempting Move:\n";
    MyString str3 = std::move(str1); // Should trigger Move Constructor

    std::cout << "\nFinal States:\n";
    std::cout << "str1: "; str1.print(); // Should be empty
    std::cout << "str2: "; str2.print(); // Should have "Hello Backend World"
    std::cout << "str3: "; str3.print(); // Should have "Hello Backend World"

    return 0;
}
```

Week 4 : Professional Build Systems (CMake) and Project Structure.

Day 1: Demystifying the Build Pipeline

When you run `g++`, four distinct things happen under the hood. You must understand these to

debug linking errors in large projects.

1. **Preprocessor:** Expands macros (`#define`) and copies the contents of header files (`#include`) directly into the `.cpp` file.
2. **Compiler:** Converts the expanded C++ code into Assembly language.
3. **Assembler:** Converts Assembly into machine code Object files (`.o` or `.obj`).
4. **Linker:** Stitches multiple Object files together into the final executable.

Day 2: Header Files & Include Guards

In multi-file projects, you separate the *Declaration* (what a function does) from the *Definition* (how it does it).

- **`.hpp` or `.h` (Header):** Contains function signatures, class layouts, and struct definitions.
- **`.cpp` (Source):** Contains the actual executable logic.

The "Multiple Definition" Trap: If two `.cpp` files include the same header, the Linker will panic because it sees the same function defined twice.

Fix: Always use Include Guards at the very top of your header files.

```
C++

#pragma once // Modern, faster way

// OR the legacy way:

#ifndef MY_HEADER_H

#define MY_HEADER_H

// ... declarations ...

#endif
```

Day 3: Namespaces & Scope Pollution

In competitive programming, typing `using namespace std;` is a reflex. In systems engineering, putting `using namespace std;` inside a header file is a critical error.

If you put it in a header, *every* file that includes that header is forced into the `std` namespace, potentially causing massive naming collisions (e.g., your custom `count` variable clashing with `std::count`).

- **Rule:** Never use `using namespace std;` in a `.hpp` file. Use explicit `std::` prefixes.

Day 4: CMake Basics (`CMakeLists.txt`)

CMake is not a compiler; it is a build system generator. It reads a `CMakeLists.txt` file and generates the exact Makefiles or Ninja build scripts needed for your specific OS.

A standard backend project structure looks like this:

```
Plaintext
```

```
/my_backend
├── CMakeLists.txt
├── /src
│   ├── main.cpp
│   └── logic.cpp
├── /include
│   └── logic.hpp
```

The Weekend Assignment: "The Polyglot Build"

Task: We are going to bridge your two worlds. We will use your exact CP template for the `main.cpp` entry point, but we will strip the actual logic out of it. The algorithm will live in a separate, modern, namespace-safe library that we will link together using CMake.

Here is the problem framing to test your new build system:

Given an array of integers, filter out all odd numbers, square the remaining even numbers, and return the sum.

1. The Build File (`CMakeLists.txt`)

Create this file in your root directory. It tells the compiler how to stitch your project together.

```
CMake
```

```
cmake_minimum_required(VERSION 3.10)

project(SystemEngine VERSION 1.0)
```

```
# Specify modern C++ standards
set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED True)

# Tell CMake where to look for header files
include_directories(include)

# Create a static library from our modern C++ logic file
add_library(MathLogic src/logic.cpp)

# Create the final executable from your CP template
add_executable(a.out src/main.cpp)

# Link the modern library into the CP executable
target_link_libraries(a.out MathLogic)
```

2. The Header (include/logic.hpp)

Strictly modern C++. No global namespaces.

C++

```
#pragma once
#include <vector>

namespace BackendSystem {
    // Declaration only
    long long process_data(const std::vector<long long>& vec);
```

```
}
```

3. The Source (src/logic.cpp)

```
C++

#include "logic.hpp"

#include <numeric>


namespace BackendSystem {

    // Definition

    long long process_data(const std::vector<long long>& vec) {

        long long res = 0;

        for(const auto& val : vec) {

            if (val % 2 == 0) {

                long long tmp = val * val;

                res += tmp;

            }

        }

        return res;

    }

}
```

4. The Entry Point (src/main.cpp)

Using your template to ingest the input, but passing the work to the linked library.

```
C++

/**
```

```
 * author: Anicetus_7
 * created: [ 2026-03-07 18:37:00 ]
**/
#include <bits/stdc++.h>
#include "logic.hpp" // Linking our modern C++ header
using namespace std;
#define int long long
#define INF (int)1e18
#define MOD (int)(1e9 + 7)
#define MAX (int)(20005)
mt19937_64 RNG(chrono::steady_clock::now().time_since_epoch().count());

void Solve(){
    int cnt;
    cin >> cnt;
    vector<int> vec(cnt);
    for(int i = 0; i < cnt; i++){
        cin >> vec[i];
    }

    // Calling the function from our compiled external library
    int res = BackendSystem::process_data(vec);
    cout << res << "\n";
}

//|----------------------------------------[MAIN]----------------------------------------|
```

```
int32_t main(){
  auto begin = std::chrono::high_resolution_clock::now();
  ios::sync_with_stdio(0); cin.tie(0);
  int t = 1;
  cin>> t;
  for(int i = 1; i <= t; i++)
   {
      //cout << "Case #" << i << ": \n";
      Solve();
   }
  auto end = std::chrono::high_resolution_clock::now();
  auto elapsed = std::chrono::duration_cast<std::chrono::nanoseconds>(end - begin);
  cerr << "Time measured: " << elapsed.count() * 1e-9 << " seconds.\n";
  return 0;
}
```

Testing the Workflow

input.in

```
Plaintext
2
4
1 2 3 4
5
10 11 12 13 14
```

expected output.out

Plaintext

```
20

440
```

To build and run this without your shell script, you would open your VSCode terminal and type:

Bash

```
mkdir build

cd build

cmake ..

make

./a.out < ../input.in > ../output.out
```